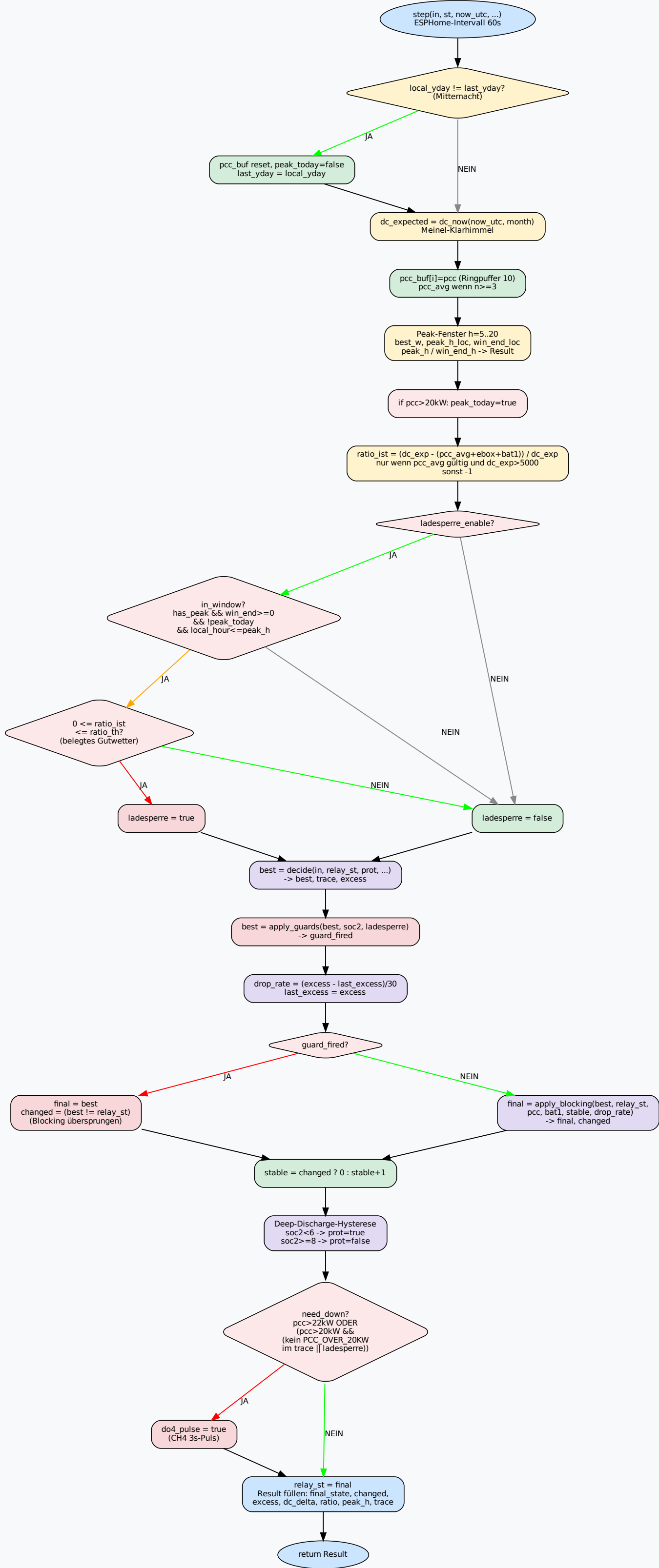


fox2db v3.3.30 — Waveshare ESP32-S3 6CH — Architektur (fox2db_logic.h, autonom auf dem ESP)

fox2db v3.3.30 — step() Gesamtablauf (alle 60s, fox2db_logic.h)



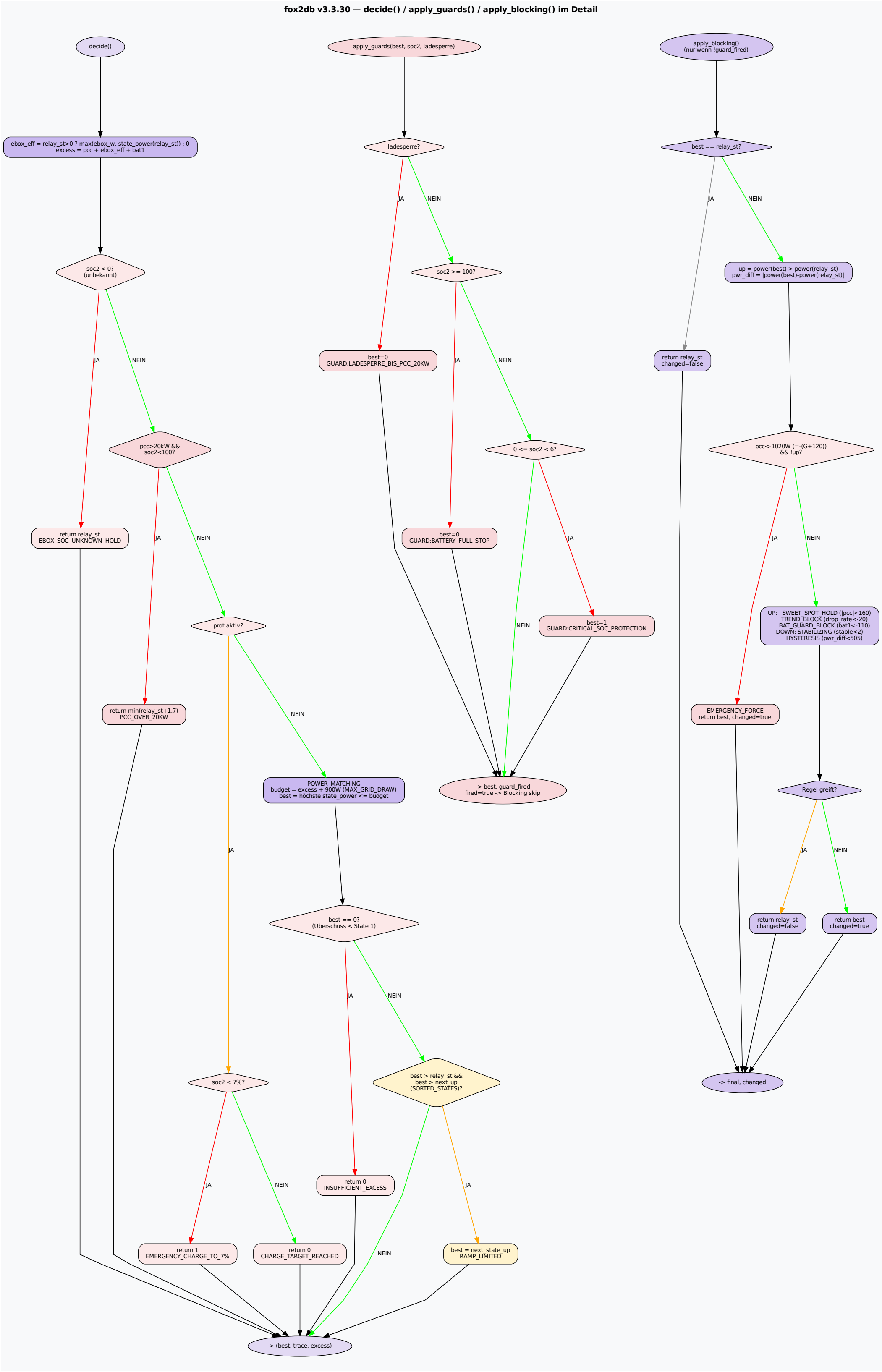
```

graph TD
    subgraph "fox2db v3.3.30 — decide() / apply_guards() / apply_blocking() im Detail"
        direction TB
        subgraph "decide()"
            D1([decide()]) --> D1_P1[ebox_eff = relay_st>0 ? max(ebox_w, state.power(relay_st)) : 0  
excess = pcc + ebox_eff + bat1]
            D1_P1 --> D1_D1{soc2 < 0?  
(unbekannt)}
            D1_D1 -- JA --> D1_P2([return relay_st  
EBOX_SOC_UNKNOWN_HOLD])
            D1_D1 -- NEIN --> D1_D2{pcc>20kW &&  
soc2<100?}
            D1_D2 -- JA --> D1_P3([return min(relay_st+1,7)  
PCC_OVER_20KW])
            D1_D2 -- NEIN --> D1_D3{prot aktiv?}
            D1_D3 -- JA --> D1_D4{soc2 < 7%?}
            D1_D3 -- NEIN --> D1_P4[POWER_MATCHING  
budget = excess + 900W (MAX_GRID_DRAW)  
best = höchste state_power <= budget]
            D1_D4 -- JA --> D1_P5([return 1  
EMERGENCY_CHARGE_TO_7%])
            D1_D4 -- NEIN --> D1_P6[return 0  
CHARGE_TARGET_REACHED]
            D1_P4 --> D1_D5{best == 0?  
(Überschuss < State 1)}
            D1_D5 -- JA --> D1_P7([return 0  
INSUFFICIENT_EXCESS])
            D1_D5 -- NEIN --> D1_D6{best > relay_st &&  
best > next_up  
(SORTED_STATES)?}
            D1_D6 -- JA --> D1_P8([best = next_state_up  
RAMP_LIMITED])
            D1_P2 --> D1_P9([-> (best, trace, excess)])
            D1_P3 --> D1_P9
            D1_P5 --> D1_P9
            D1_P6 --> D1_P9
            D1_P7 --> D1_P9
            D1_P8 --> D1_P9
        end

        subgraph "apply_guards(best, soc2, ladesperre)"
            D2([apply_guards(best, soc2, ladesperre)]) --> D2_D1{ladesperre?}
            D2_D1 -- JA --> D2_P1([best=0  
GUARD:LADESPERRE_BIS_PCC_20KW])
            D2_D1 -- NEIN --> D2_D2{soc2 >= 100?}
            D2_D2 -- JA --> D2_P2([best=0  
GUARD:BATTERY_FULL_STOP])
            D2_D2 -- NEIN --> D2_D3{0 <= soc2 < 6?}
            D2_D3 -- JA --> D2_P3([best=1  
GUARD:CRITICAL_SOC_PROTECTION])
            D2_D3 -- NEIN --> D2_P4([-> best, guard_fired  
fired=true -> Blocking skip])
            D2_P1 --> D2_P4
            D2_P2 --> D2_P4
            D2_P3 --> D2_P4
        end

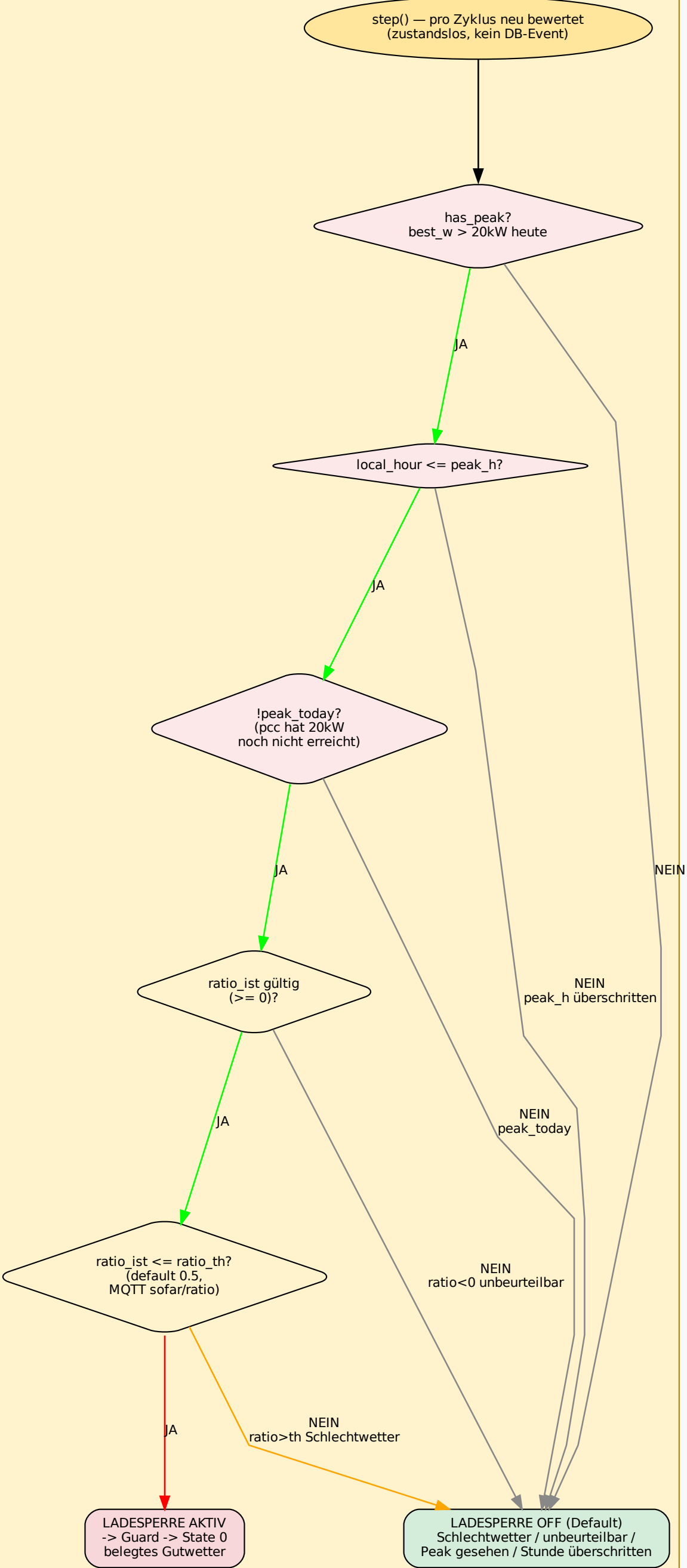
        subgraph "apply_blocking!(nur wenn !guard_fired)"
            D3([apply_blocking!  
(nur wenn !guard_fired)]) --> D3_D1{best == relay_st?}
            D3_D1 -- JA --> D3_P1([return relay_st  
changed=false])
            D3_D1 -- NEIN --> D3_P2[up = power(best) > power(relay_st)  
pwr_diff = |power(best)-power(relay_st)|]
            D3_P2 --> D3_D2{pcc<-1020W (=-G+120))  
&& !up?}
            D3_D2 -- JA --> D3_P3[EMERGENCY_FORCE  
return best, changed=true]
            D3_D2 -- NEIN --> D3_D3{Regel greift?}
            D3_D3 -- JA --> D3_P4([return relay_st  
changed=false])
            D3_D3 -- NEIN --> D3_P5([return best  
changed=true])
            D3_P1 --> D3_P6([-> final, changed])
            D3_P3 --> D3_P6
            D3_P4 --> D3_P6
            D3_P5 --> D3_P6
        end
    end

```



fox2db v3.3.30 — LADESPERRE-Logik + Soyo-Entladung (RS485)

LADESPERRE — Akku morgens leer halten für >20kW-Mittagspeak



Soyo-Entladung (max 900W, alle 60s soyo/calc, RS485-TX alle 3s)

